

PCA on Soil Temperature Data

September 18, 2026

0.1 PCA on Soil Temperature Data

Parquet is a columnar storage file format optimized for use with large-scale data processing frameworks. It is efficient in terms of both storage and query performance, especially with large datasets.

```
[3]: # Importing the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pyarrow

# Importing the libraries for PCA
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
[4]: # Reading final dataset

df = pd.read_parquet('pueblo.parquet')

df.head()

# note that cfs stands for cubic feet/person

# gwl stands for ground water level
```

```
[4]:
```

	date	precip	max_temp	min_temp	gwl	sm_20cm	st_20cm	\
85	2010-02-14	0.0	36	13	20.52	0.205500	0.408333	
86	2010-02-15	0.0	41	7	20.51	0.215333	0.595833	
87	2010-02-16	0.0	43	8	20.51	0.210417	0.500000	
88	2010-02-17	0.0	51	10	20.51	0.209708	0.500000	
89	2010-02-18	0.0	51	16	20.50	0.210542	0.637500	

	sm_50cm	st_50cm	sm_100cm	st_100cm	daily_discharge_cfs
85	0.182875	1.320833	0.135333	3.700000	136.0
86	0.183292	1.445833	0.137250	3.625000	129.0
87	0.183625	1.495833	0.136583	3.700000	130.0
88	0.184667	1.437500	0.135625	3.645833	132.0

89 0.185125 1.479167 0.135333 3.708333 135.0

```
[5]: num_rows, num_cols = df.shape

print(f"Number of rows: {num_rows}")
print(f"Number of columns: {num_cols}")
```

Number of rows: 4881
Number of columns: 12

```
[6]: # We can see that we have 3 different type of soil temperature data.
# With PCA we aim to reduce the dimensionality of the data, here particularly
↳ the soil temperature data.

# creating feature list for soil moisture data
pca_st_list = ['st_20cm', 'st_50cm', 'st_100cm']

X = df[pca_st_list]

X
```

```
[6]:      st_20cm  st_50cm  st_100cm
85      0.408333  1.320833  3.700000
86      0.595833  1.445833  3.625000
87      0.500000  1.495833  3.700000
88      0.500000  1.437500  3.645833
89      0.637500  1.479167  3.708333
...
5039  18.787500  19.025000  19.116667
5040  18.779167  19.033333  19.108333
5041  19.100000  19.041667  19.095833
5042  18.854167  19.250000  19.033333
5043  18.491667  19.408333  19.016667
```

[4881 rows x 3 columns]

Scaling is a crucial step in Principal Component Analysis (PCA) for several reasons. Here's why scaling (or standardizing) the data is important before applying PCA:

1. *Equal Weighting of Features*

Variance Differences: Different features in your dataset may have different units and scales (e.g., height in centimeters vs. weight in kilograms). If the features are not scaled to a common range, those with larger scales will dominate the principal components. This can lead to misleading results, where the principal components reflect the scale of the features rather than their actual importance or relationships. **Standardization:** Scaling (typically standardizing) transforms the data so that each feature has a mean of 0 and a standard deviation of one. This ensures that all features contribute equally to the analysis.

2. Improving PCA Performance

Covariance Matrix: PCA involves the calculation of the covariance matrix of the features. If the features have different scales, the covariance matrix will be dominated by features with larger variances, skewing the principal components towards those features. **Numerical Stability:** Scaling the data can also help improve the numerical stability and accuracy of the PCA computation, as large differences in scale can lead to numerical issues during matrix operations.

3. Interpretability

Consistent Units: By scaling the data, all features are transformed to a common scale, making the results of PCA easier to interpret. The principal components then reflect the underlying relationships between features rather than their scale.

```
[7]: # Standardizing the features
X_scaled = StandardScaler().fit_transform(X)
X_scaled

[7]: array([[ -1.50143206, -1.60682967, -1.68986976],
          [-1.48106444, -1.59132543, -1.70192544],
          [-1.49147456, -1.58512373, -1.68986976],
          ...,
          [ 0.52899334,  0.59115548,  0.78489333],
          [ 0.50228913,  0.61699589,  0.77484693],
          [ 0.46291173,  0.6366346 ,  0.77216789]])
```

0.1.1 Explained Variance in PCA

PCA is a dimensionality reduction technique that transforms the data into a new coordinate system where the axes (principal components) are ordered by the amount of variance they capture from the original data. Here's what explained variance signifies:

0.1.2 Variance Explained by Each Component:

Explained variance measures how much of the total variability (or variance) in the data is captured by each principal component. For each principal component, the explained variance quantifies the proportion of the total variance that this component captures.

0.1.3 Mathematical Aspect:

In PCA, each principal component is a linear combination of the original features. The variance of each component is derived from the eigenvalues of the covariance matrix of the original data. The explained variance for a principal component is proportional to its corresponding eigenvalue. Cumulative Explained Variance:

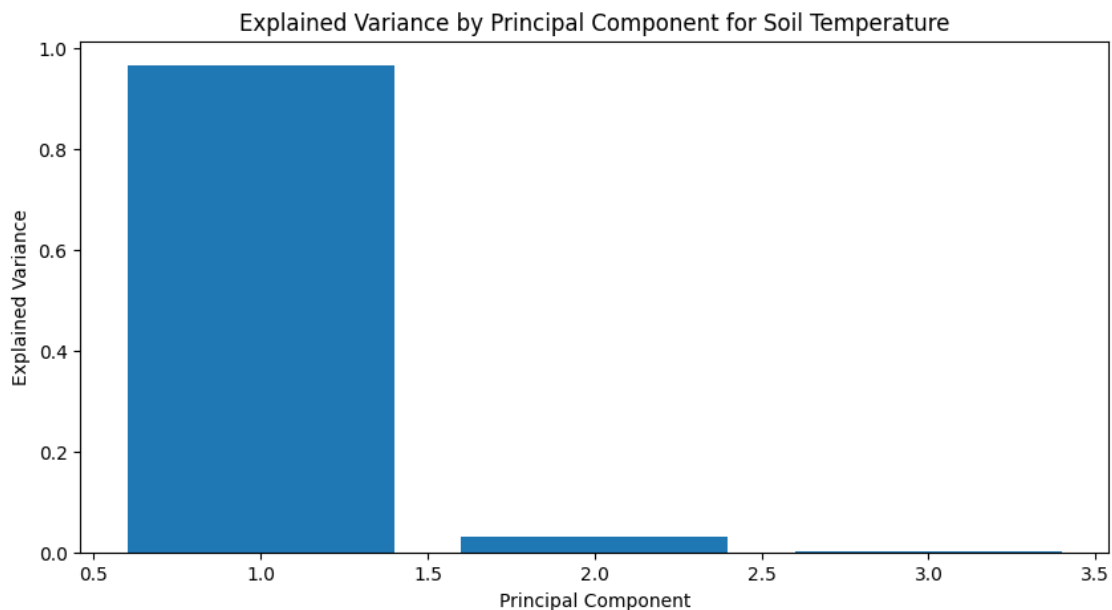
Cumulative explained variance refers to the sum of the explained variances of the principal components up to a certain point. It helps to determine how many principal components are needed to capture a desired amount of the total variance in the data.

```
[8]: # Performing PCA with 3 components and plotting the explained variance

pca = PCA(n_components=3)
principalComponents = pca.fit_transform(X_scaled)
principalDf = pd.DataFrame(data = principalComponents
                           , columns = ['principal component 1', 'principal component 2',
                                       ↪ 'principal component 3'])

# Plotting the explained variance as bar plot

plt.figure(figsize=(10,5))
plt.bar(range(1,4), pca.explained_variance_ratio_)
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance')
plt.title('Explained Variance by Principal Component for Soil Temperature')
plt.show()
```



We can see from the plots, the first principal component explains more than 90% of the variance for soil temperature data

```
[9]: # We can use the first principal component to represent the soil temperature
      ↪ data.

# Adding the first principal component to the dataframe
df['st_pca'] = principalDf['principal component 1']
```

```
[10]: # Resetting the index and dropping the old index
df.reset_index(drop=True, inplace=True)
```

```
[ ]: # Saving the final dataframe with PCA data

# This is if you want to save it

# df.to_parquet('data/final_data/pueblo_pca.parquet')
```

For your programming assignment, you will be repeating this exercise but this time using Soil Moisture data

```
[ ]:
```